HADROS3: Physics and Statistical Theory of the Implemented Pipeline

Rafael C. R. de Lima

Implemented theory through H3-W9a POWHEG dry-run preparation

June 26, 2026

This document describes only the physics, statistics, numerical methods, and proxy models implemented in the current HADROS3 pipeline. It is not a user manual, dashboard guide, installation guide, or proposal for future stages. Where the implementation uses diagnostic renderers or physics proxies, that status is stated explicitly. This is a living theory document for HADROS3. It describes exclusively the physics currently implemented in the official HADROS3 pipeline. Future or planned models are not described here until they become part of the official implementation.

Contents

Scope and Notation **3**

1 Camera / Kerr Ray Tracing **5**

1.1 Implemented camera modes 5

1.2 Camera frame 5

1.3 Pinhole diagnostic projection 6

1.4 Inputs and outputs 6

2 Black Hole and Kerr Geometry **7**

2.1 Metric functions 7

2.2 ZAMO quantities 8

2.3 Null invariant 8

3 Torus / Medium **9**

3.1 Analytic density model 9

3.2 MediumRenderer 9

3.3 Inputs and outputs 10

4 UHE Source **11**

4.1 Coordinate cone sampling 11

4.2 Energy and directions 11

4.3 Inputs and outputs 12

5 Forward Geodesics **13**

5.1 Momentum construction 13

5.2 Hamiltonian system 13

5.3 Segment quantities 14

5.4 Inputs and outputs 14

6 DIS Interaction Sampler **15**

6.1 Density and local energy 15

6.2 Cross-section interpolation . . . 15
6.3 Optical depth and probability . . . 15
6.4 Interaction sampling . . . 16
6.5 Weights . . . 16
6.6 Inputs and outputs . . . 16

7 Observer Bridge **18**
7.1 Purpose . . . 18
7.2 Geometry proxies . . . 18
7.3 Score definition . . . 18
7.4 Proxy risk flags . . . 19
7.5 Inputs and outputs . . . 19

8 POWHEG Dry Run / Hard Process Preparation **20**
8.1 Dry-run role . . . 20
8.2 Candidate selection . . . 20
8.3 Input-card mapping . . . 21
8.4 Event requests . . . 21
8.5 Inputs and outputs . . . 21

9 Provenance and Statistical Weights **22**
9.1 Stage provenance . . . 22
9.2 Weight chain . . . 22
9.3 Random seeds . . . 23

10 Pipeline Summary **24**
10.1 Implemented stage contract . . . 24
10.2 Explicit non-goals of the implemented stages . . . 24

A Appendix: Symbols and Product Names **26**

Scope and Notation

The implemented HADROS3 pipeline is organized as a staged calculation,

$$\text{inputs} \rightarrow \text{run}() \rightarrow \text{outputs} \rightarrow \text{diagnostics} \rightarrow \text{provenance}. \tag{0.1}$$

Each implemented stage consumes official products from earlier stages and writes to its own output directory. The stages covered here are the camera preview, Kerr geometry utilities, analytic medium, UHE source, forward null geodesics, DIS optical-depth sampler, Observer Bridge scoring, and POWHEG dry-run hard-process preparation.

This document is maintained as a required living reference for implemented physics. When a new HADROS3 stage changes or adds physical equations, statistical weights, physical approximations, or proxy models, this source file and the regenerated PDF must be updated with the implementation.

Table 1: Implemented-stage coverage in this theory document. Planned stages are listed only to mark that their physics is not yet documented as implemented.

HADROS3 stage	Implementation status	Theory documented
H3-W5 UHE Source	implemented	yes
H3-W6 Forward Geodesics	implemented	yes
H3-W7 DIS Interaction Sampler	implemented	yes
H3-W8 Observer Bridge	implemented/proxy	yes
H3-W9a POWHEG Dry Run	implemented/dry run	yes
H3-W9b POWHEG Real Run	planned	no
H3-W10 PYTHIA	planned	no
H3-W11 GEANT4	planned	no
H3-W12 Photon Transport	planned	no
H3-W13 Spectra	planned	no

All radii in the dynamical Kerr calculation are expressed in gravitational radius units,

$$r_g = \frac{GM}{c^2}. \tag{0.2}$$

The implementation uses Boyer-Lindquist coordinates $x^\mu = (t, r, \theta, \phi)$ and the metric signature $(-,+,+,+)$. The dimensionless Kerr spin is denoted by a in units of r_g. The code clamps spin-dependent operations away from the extremal singular limit where needed.

Table 2: Primary symbols used in the implemented theory.

Symbol	Meaning
r, θ, ϕ	Boyer-Lindquist spherical coordinates in r_g units.
Σ, Δ, A	Standard Kerr metric functions.
p_μ	Covariant four-momentum components.
$E_{\nu,\mathrm{local}}$	Neutrino energy measured in a local medium or ZAMO frame.
$\rho(r, \theta)$	Analytic torus rest-mass density.
n_b	Baryon number density, $n_b = \rho/m_b$.
$\sigma_{\nu N}(E)$	Tabulated charged-current neutrino-nucleon DIS cross section.
τ	Integrated DIS optical depth along one forward path.
S_{obs}	Observer Bridge final observation score.

Chapter 1

Camera / Kerr Ray Tracing

1.1 Implemented camera modes

The camera preview layer is a visual and diagnostic stage. It is separate from the DIS interaction physics and from the Observer Bridge scoring. Its main product is a rendered observer-view image, typically `CameraPreview/hadros3_camera_preview.png`. The implemented backend selection includes:

- `analytic_geometry_only`: always available; a schematic camera preview, not a physical Kerr ray-traced image.
- `kerr_like_cuda`: a self-contained HADROS3 CUDA preview executable using a Kerr-like camera model.
- `full_kerr`: a self-contained HADROS3 CUDA preview executable using the full-Kerr preview mode when available.
- `legacy_cpu_geodesic_preview`: detected for provenance but not used as the primary self-contained HADROS3 runtime path.

The camera preview can render a torus-like visual object and polar funnels. This visual object is not the DIS optical-depth calculation. The provenance marks camera-preview products separately from the shared `MediumRenderer`. In current provenance, the camera preview records `medium_renderer_used = false` because its purpose is visual camera preview, not the shared diagnostic density-map renderer.

1.2 Camera frame

Observer Bridge camera diagnostics use the same geometric convention for the camera direction that is also relevant when overlaying candidates on the camera preview. A camera position is built from

$$\mathbf{x}_{\mathrm{obs}} = r_{\mathrm{obs}} \begin{pmatrix} \sin i \cos \varphi_{\mathrm{obs}} \\ \sin i \sin \varphi_{\mathrm{obs}} \\ \cos i \end{pmatrix}, \tag{1.1}$$

where r_{obs} is `observer_distance_rg`, i is `observer_inclination_deg`, and φ_{obs} is `observer_azimuth_deg`. The forward axis points approximately toward the origin,

$$\hat{\mathbf{f}} = -\frac{\mathbf{x}_{\mathrm{obs}}}{|\mathbf{x}_{\mathrm{obs}}|}. \tag{1.2}$$

The right and up axes are built from a world z axis, with a fallback right axis if the camera is too close to the polar direction:

$$\hat{\mathbf{r}}_{\mathrm{cam}} = \frac{\hat{\mathbf{f}} \times \hat{\mathbf{z}}}{|\hat{\mathbf{f}} \times \hat{\mathbf{z}}|}, \tag{1.3}$$

$$\hat{\mathbf{u}}_{\mathrm{cam}} = \frac{\hat{\mathbf{r}}_{\mathrm{cam}} \times \hat{\mathbf{f}}}{|\hat{\mathbf{r}}_{\mathrm{cam}} \times \hat{\mathbf{f}}|}. \tag{1.4}$$

1.3 Pinhole diagnostic projection

For Observer Bridge candidate overlays, the implemented projection is a geometric pinhole proxy. It converts a candidate position $\mathbf{x}$ into a camera vector

$$\mathbf{d} = \mathbf{x} - \mathbf{x}_{\mathrm{obs}} \tag{1.5}$$

and projects it as

$$z_{\mathrm{cam}} = \mathbf{d} \cdot \hat{\mathbf{f}}, \tag{1.6}$$

$$x_{\mathrm{ndc}} = \frac{\mathbf{d} \cdot \hat{\mathbf{r}}_{\mathrm{cam}}}{z_{\mathrm{cam}} \tan(\mathrm{FOV}_x/2)}, \tag{1.7}$$

$$y_{\mathrm{ndc}} = \frac{\mathbf{d} \cdot \hat{\mathbf{u}}_{\mathrm{cam}}}{z_{\mathrm{cam}} \tan(\mathrm{FOV}_y/2)}. \tag{1.8}$$

The field-of-view flag is true when $z_{\mathrm{cam}} > 0$ and $|x_{\mathrm{ndc}}| \leq 1$, $|y_{\mathrm{ndc}}| \leq 1$. The camera overlay records `candidate_overlay_projection_model = geometric_pinhole_proxy` and `candidate_overlay_not_r = true`. Therefore the background image may be generated by the camera renderer, but the plotted Observer Bridge candidates are not ray-traced secondary particles.

1.4 Inputs and outputs

Role	Implemented item
Inputs	Black-hole spin, observer distance, inclination, azimuth, FOV, image resolution, torus and funnel visual parameters.
Outputs	`CameraPreview/hadros3_camera_preview.png`, `CameraPreview/hadros3_camera_preview_summary.json`.
Approximations	Analytic fallback is schematic. CUDA preview is visual camera rendering; candidate overlays remain pinhole proxies.
Provenance	Records backend used, fallback status, whether CUDA was used, and whether full-Kerr preview mode was requested.

Chapter 2

Black Hole and Kerr Geometry

2.1 Metric functions

HADROS3 uses the Kerr metric in Boyer-Lindquist coordinates and in r_g units. The standard metric functions are

$$\Sigma = r^2 + a^2 \cos^2\theta, \tag{2.1}$$
$$\Delta = r^2 - 2r + a^2, \tag{2.2}$$
$$A = (r^2 + a^2)^2 - a^2 \Delta \sin^2\theta. \tag{2.3}$$

The outer horizon radius is

$$r_+ = 1 + \sqrt{1 - a^2}. \tag{2.4}$$

The nonzero covariant metric components implemented for the dynamical calculation are

$$g_{tt} = -\left(1 - \frac{2r}{\Sigma}\right), \tag{2.5}$$
$$g_{t\phi} = -\frac{2ar\sin^2\theta}{\Sigma}, \tag{2.6}$$
$$g_{rr} = \frac{\Sigma}{\Delta}, \tag{2.7}$$
$$g_{\theta\theta} = \Sigma, \tag{2.8}$$
$$g_{\phi\phi} = \left(r^2 + a^2 + \frac{2a^2 r \sin^2\theta}{\Sigma}\right)\sin^2\theta = \frac{A\sin^2\theta}{\Sigma}. \tag{2.9}$$

The corresponding inverse components used by the Hamiltonian geodesic solver are

$$g^{tt} = -\frac{A}{\Sigma\Delta}, \tag{2.10}$$
$$g^{t\phi} = -\frac{2ar}{\Sigma\Delta}, \tag{2.11}$$
$$g^{rr} = \frac{\Delta}{\Sigma}, \tag{2.12}$$
$$g^{\theta\theta} = \frac{1}{\Sigma}, \tag{2.13}$$
$$g^{\phi\phi} = \frac{\Delta - a^2\sin^2\theta}{\Sigma\Delta\sin^2\theta}. \tag{2.14}$$

2.2 ZAMO quantities

The implemented ZAMO lapse and frame-dragging angular velocity are

$$\alpha = \sqrt{\frac{\Sigma\Delta}{A}}, \tag{2.15}$$

$$\omega = \frac{2ar}{A}. \tag{2.16}$$

For a covariant momentum with components p_t and p_ϕ, the ZAMO energy proxy used in the C++ tetrad utilities is

$$E_{\mathrm{ZAMO}} = -\frac{p_t + \omega p_\phi}{\alpha}. \tag{2.17}$$

The local tetrad initialization maps a normalized local spatial direction (n_r, n_θ, n_ϕ) into a contravariant null vector and then lowers the index with $g_{\mu\nu}$. The implementation checks that the resulting null norm is finite and that the ZAMO energy is positive.

2.3 Null invariant

The null condition is

$$g^{\mu\nu} p_\mu p_\nu = 0. \tag{2.18}$$

Both Python reference utilities and the C++ forward-geodesic backend monitor this invariant. The calculation also monitors the Killing quantities

$$E_\infty = -p_t, \tag{2.19}$$

$$L_z = p_\phi, \tag{2.20}$$

which are conserved because the metric is stationary and axisymmetric.

Chapter 3

Torus / Medium

3.1 Analytic density model

The implemented medium model is an analytic torus density field. It is not a hydrodynamical simulation. The density is evaluated as

$$\rho(r,\theta) = \rho_0 \exp\left[-\frac{1}{2}\left(\frac{r - r_{\mathrm{peak}}}{\sigma_r}\right)^2\right] \exp\left[-\frac{1}{2}\left(\frac{\theta - \pi/2}{\sigma_\theta}\right)^2\right], \tag{3.1}$$

with the hard radial support

$$\rho(r,\theta) = 0 \quad \text{if} \quad r < r_{\mathrm{inner}} \quad \text{or} \quad r > r_{\mathrm{outer}}. \tag{3.2}$$

The implemented radial width is

$$\sigma_r = \frac{1}{2}(r_{\mathrm{outer}} - r_{\mathrm{inner}}), \tag{3.3}$$

and the angular width is

$$\sigma_\theta = \max(\theta_{\mathrm{half}}, 10^{-6}). \tag{3.4}$$

The configuration parameter `half_opening_angle_deg` is therefore a Gaussian angular width, not a hard angular boundary. This interpretation is recorded in diagnostics as `half_opening_angle_interpreta = gaussian_width_not_boundary`.

Within the hard radial support, the code can apply a configured `density_floor_g_cm3` by taking the maximum of the analytic value and the floor. Outside the radial shell, the density remains exactly zero.

3.2 MediumRenderer

The shared `MediumRenderer` renders the same analytic density model used by the DIS sampler diagnostics. It records

$$\texttt{medium\_renderer\_used} = \texttt{true}, \tag{3.5}$$

$$\texttt{density\_model\_has\_hard\_radial\_cut} = \texttt{true}, \tag{3.6}$$

$$\texttt{density\_model\_theta\_profile} = \texttt{gaussian}, \tag{3.7}$$

$$\texttt{density\_model\_theta\_is\_hard\_cut} = \texttt{false}. \tag{3.8}$$

The renderer draws the hard radial cuts and Gaussian angular density levels. Those angular levels are visual density contours, not physical walls.

3.3 Inputs and outputs

Role	Implemented item
Inputs	`rho0_g_cm3`, `r_inner_rg`, `r_peak_rg`, `r_outer_rg`, `half_opening_angle_deg`, optional density floor.
Used by	DIS density evaluation, DIS density diagnostics, forward geometry diagnostics, Observer Bridge diagnostic projections.
Approximation	Static analytic density field; no GRMHD evolution; no self-consistent thermal or composition model.
Output semantics	Positive density only inside the hard radial shell; Gaussian tail in θ.

Chapter 4

UHE Source

4.1 Coordinate cone sampling

The implemented UHE source is a coordinate-volume sampler in a polar cone. For a cone spanning $r_{\min} \le r \le r_{\max}$ and $\theta_{\min} \le \theta \le \theta_{\max}$, the coordinate volume used for sampling is

$$V_{\text{cone}} = \frac{2\pi}{3}\left(r_{\max}^3 - r_{\min}^3\right)\left(\cos\theta_{\min} - \cos\theta_{\max}\right). \tag{4.1}$$

Uniform coordinate-volume sampling is implemented by drawing $u_r, u_\theta, u_\phi \sim U(0,1)$ and setting

$$r = \left[r_{\min}^3 + u_r(r_{\max}^3 - r_{\min}^3)\right]^{1/3}, \tag{4.2}$$
$$\cos\theta = \cos\theta_{\min} - u_\theta(\cos\theta_{\min} - \cos\theta_{\max}), \tag{4.3}$$
$$\phi = 2\pi u_\phi. \tag{4.4}$$

The source sampling and physical PDFs are both recorded as

$$p_{\text{source}} = \frac{1}{V_{\text{cone}}}, \tag{4.5}$$

so the implemented source weight is unity for the default model:

$$w_{\text{source}} = \frac{p_{\text{physical}}}{p_{\text{sampling}}} = 1. \tag{4.6}$$

4.2 Energy and directions

The implemented source energy model is monoenergetic:

$$E_{\nu,\text{emit}} = E_0. \tag{4.7}$$

For the isotropic local direction model, the code samples a unit vector in a ZAMO orthonormal frame:

$$\cos\alpha = 2u_\alpha - 1, \tag{4.8}$$
$$\beta = 2\pi u_\beta, \tag{4.9}$$
$$n_r = \cos\alpha, \tag{4.10}$$
$$n_\theta = \sin\alpha\cos\beta, \tag{4.11}$$
$$n_\phi = \sin\alpha\sin\beta. \tag{4.12}$$

The sampling PDF is

$$p_\Omega = \frac{1}{4\pi}, \tag{4.13}$$

and the default direction weight is unity.

The source stage does not itself produce the physical Kerr four-momentum for forward propagation. It records the source position, energy, and local direction. The forward-geodesic stage constructs the physical Kerr null momentum from those quantities.

4.3 Inputs and outputs

Role	Implemented item
Inputs	Cone geometry, sample count, random seed, monoenergetic neutrino energy, direction model.
Outputs	`UHEsource/uhe_neutrino_source_samples.jsonl` and source summary products.
Weights	`source_weight = 1` and `direction_weight = 1` for the implemented default uniform/properly matched sampling.
Approximation	Coordinate cone volume, not a GR invariant emissivity integral.

Chapter 5

Forward Geodesics

5.1 Momentum construction

The forward stage consumes UHE source samples and constructs a Kerr null covector. For a local isotropic direction, the ZAMO tetrad maps the local direction into a null four-vector and then into covariant momentum components. For legacy coordinate radial directions, the implementation constructs a covariant momentum with $p_t = -E$ and solves the null constraint for p_r.

The implementation records that the physical Kerr four-momentum is constructed in the forward stage, not in the source sampler.

5.2 Hamiltonian system

The Hamiltonian for null geodesics is

$$H(x,p) = \frac{1}{2} g^{\mu\nu}(x) p_\mu p_\nu. \tag{5.1}$$

The implemented equations of motion are

$$\frac{\mathrm{d}x^\mu}{\mathrm{d}\lambda} = g^{\mu\nu} p_\nu, \tag{5.2}$$

$$\frac{\mathrm{d}p_i}{\mathrm{d}\lambda} = -\frac{1}{2} \partial_i g^{\mu\nu} p_\mu p_\nu, \qquad i \in \{r, \theta\}. \tag{5.3}$$

The cyclic momenta are conserved by construction in the differential system:

$$\frac{\mathrm{d}p_t}{\mathrm{d}\lambda} = 0, \qquad \frac{\mathrm{d}p_\phi}{\mathrm{d}\lambda} = 0. \tag{5.4}$$

The current implementation uses an RK4 stepping procedure with substeps and invariant checks. Derivatives of inverse metric components are evaluated numerically. When needed, p_r can be renormalized to keep the null constraint within tolerance. The solver stops when it reaches the horizon neighborhood, the configured outer radius, maximum steps, or an invalid invariant condition.

5.3 Segment quantities

Each accepted forward path is written as a set of path segments. The segment length used downstream is a coordinate path-length proxy,

$$\Delta l_{\mathrm{coord}} = \left[(\Delta r)^2 + (r_{\mathrm{mid}}\Delta\theta)^2 + (r_{\mathrm{mid}}\sin\theta_{\mathrm{mid}}\Delta\phi)^2\right]^{1/2}. \tag{5.5}$$

This is then converted to cm in the DIS stage by multiplying by r_g in cm. This is an implemented optical-depth path-length approximation and should not be confused with a fully covariant matter-frame line element.

5.4 Inputs and outputs

Role	Implemented item
Inputs	`UHEsource/uhe_neutrino_source_samples.jsonl`, black-hole spin, step settings, tolerances, outer radius, requested number of samples.
Outputs	`ForwardGeodesics/uhe_neutrino_forward_paths.jsonl`, `ForwardGeodesics/uhe_neutrino_forward_path_segments.jsonl`, and summary diagnostics.
Invariants	Null norm, Killing energy, and L_z are monitored.
Approximation	Numerical RK4 Hamiltonian evolution with coordinate segment lengths for downstream optical depth.

Chapter 6

DIS Interaction Sampler

6.1 Density and local energy

The DIS sampler consumes forward path segments and evaluates the analytic torus density at segment midpoints. The baryon density is

$$n_b(r,\theta) = \frac{\rho(r,\theta)}{m_b}, \tag{6.1}$$

where m_b is the baryon mass in grams.

The local neutrino energy is computed from the covariant momentum. If the configured static medium model is valid at that position, the code uses the static observer expression

$$E_{\text{local}} = -p_t u^t, \qquad u^t = \frac{1}{\sqrt{-g_{tt}}}. \tag{6.2}$$

If a static observer is not valid, or if the implementation needs the fallback, the ZAMO expression is used:

$$E_{\text{local}} = -\left(\frac{p_t}{\alpha} + \frac{\omega p_\phi}{\alpha}\right) = -\frac{p_t + \omega p_\phi}{\alpha}. \tag{6.3}$$

The static-to-ZAMO fallback is recorded as a physics-risk approximation.

6.2 Cross-section interpolation

The DIS charged-current cross sections are read from compact tabulated files for the implemented GBW and IIM models. If an energy lies inside the tabulated range, the interpolation is log-log:

$$\ln \sigma_{\nu N}(E) = \ln \sigma_0 + \frac{\ln E - \ln E_0}{\ln E_1 - \ln E_0} \left(\ln \sigma_1 - \ln \sigma_0\right). \tag{6.4}$$

Requests outside the tabulated range are rejected for that segment and the segment contribution is set to zero in the diagnostic recomputation.

6.3 Optical depth and probability

For each segment,

$$\Delta\tau_i = n_{b,i}\, \sigma_{\nu N}(E_{\text{local},i})\, \Delta l_i, \tag{6.5}$$

where Δl_i is the coordinate path-length proxy converted from r_g to cm. The total optical depth along a path is

$$\tau = \sum_i \Delta\tau_i. \tag{6.6}$$

The interaction probability is

$$P_{\rm int} = 1 - \exp(-\tau), \tag{6.7}$$

implemented numerically as

$$P_{\rm int} = -\operatorname{expm1}(-\tau) \tag{6.8}$$

with clipping to the interval $[0, 1]$.

6.4 Interaction sampling

If a path is accepted by the Bernoulli trial with probability $P_{\rm int}$, the segment index is sampled by the inverse CDF

$$P(i) = \frac{\Delta\tau_i}{\sum_j \Delta\tau_j}. \tag{6.9}$$

After the segment is selected, the implemented accepted-point sampler requires the final recorded interaction point to have positive medium density. It performs rejection sampling inside the chosen segment and records

$$\texttt{interaction\_point\_density\_checked} = \texttt{true}, \tag{6.10}$$

$$\texttt{interaction\_point\_inside\_medium} = \texttt{true} \quad \text{when} \quad \rho(r_{\rm int}, \theta_{\rm int}) > 0, \tag{6.11}$$

$$\texttt{interaction\_point\_rho\_g\_cm3} = \rho(r_{\rm int}, \theta_{\rm int}). \tag{6.12}$$

The fallback policy is recorded in `interaction_point_sampling_method`. The present criterion for a valid accepted point is $\rho(r_{\rm int}, \theta_{\rm int}) > 0$.

6.5 Weights

The DIS products preserve source and direction weights from the UHE source. The pre-event weight used downstream is recorded as `final_pre_event_weight`. The DIS sampler also records optical-depth and expected-interaction quantities so that later stages can distinguish the sampling decision from pre-event observation scoring.

6.6 Inputs and outputs

Role	Implemented item
Inputs	Forward geodesic paths and segments, source samples, DIS cross-section tables, analytic medium parameters, random seed.
Outputs	`DIS/dis_path_optical_depths.jsonl`, `DIS/dis_accepted_interactions.jsonl`, optical-depth report, diagnostics, GBW/IIM comparison products.

Diagnostics	Tau distribution, interaction probability distribution, optical depth maps, medium density map, accepted interaction distributions, sigma and correlation plots.
Approximation	Static or ZAMO local energy model; coordinate path-length proxy; compact cross-section tables.

Chapter 7

Observer Bridge

7.1 Purpose

The Observer Bridge consumes accepted DIS interactions and assigns a non-destructive observation score. It does not generate secondary particles, does not call POWHEG, does not call PYTHIA, does not call GEANT4, and does not perform photon transport. All accepted DIS interactions become Observer Bridge candidates, even if they are outside the camera FOV or have small score.

7.2 Geometry proxies

The interaction position is converted to Cartesian coordinates through

$$(x, y, z) = (r \sin\theta \cos\phi,\ r \sin\theta \sin\phi,\ r \cos\theta)\,. \tag{7.1}$$

The observer is placed at the configured camera position. The geometric direction from the interaction to the observer is

$$\hat{\mathbf{n}}_{\mathrm{obs}} = \frac{\mathbf{x}_{\mathrm{obs}} - \mathbf{x}_{\mathrm{int}}}{|\mathbf{x}_{\mathrm{obs}} - \mathbf{x}_{\mathrm{int}}|}. \tag{7.2}$$

The implemented escape proxy compares this direction with the local radial direction,

$$w_{\mathrm{esc}} = \max\left(0,\ \hat{\mathbf{x}}_{\mathrm{int}} \cdot \hat{\mathbf{n}}_{\mathrm{obs}}\right). \tag{7.3}$$

This is a proxy, not a particle-transport escape calculation.

The FOV weight is either a hard FOV flag or a soft Gaussian-like angular proxy, depending on configuration. The visibility, line-of-sight, and redshift models are explicitly recorded as proxies. The implemented default redshift and line-of-sight weights are unity proxies unless configured otherwise.

7.3 Score definition

The implemented physics weight in the C++ Observer Bridge is

$$w_{\mathrm{physics}} = \texttt{final\_pre\_event\_weight}, \tag{7.4}$$

with a fallback to source times direction weight when that field is absent. The observer weight is the product of the implemented proxy factors:

$$w_{\mathrm{observer}} = w_{\mathrm{esc}}\, w_{\mathrm{visibility}}\, w_{\mathrm{FOV}}\, w_{\mathrm{distance}}\, w_{\mathrm{redshift}}\, w_{\mathrm{LOS}}. \tag{7.5}$$

The final observation score is

$$S_{\mathrm{obs}} = w_{\mathrm{physics}}\, w_{\mathrm{observer}}. \tag{7.6}$$

All three quantities are recorded for each candidate: `physics_weight`, `observer_weight`, and `final_observation_score`. The implementation enforces non-negative proxy weights by clipping or construction.

7.4 Proxy risk flags

The reports record the proxy nature of the bridge:

$$\texttt{secondary\_particle\_proxy\_model} \neq \texttt{full\_transport}, \tag{7.7}$$

$$\texttt{escape\_proxy\_physics\_risk} = \texttt{true}, \tag{7.8}$$

$$\texttt{visibility\_proxy\_physics\_risk} = \texttt{true}, \tag{7.9}$$

$$\texttt{redshift\_proxy\_physics\_risk} = \texttt{true}. \tag{7.10}$$

The stage status is scoring-only. It is intentionally non-destructive.

7.5 Inputs and outputs

Role	Implemented item
Inputs	`DIS/dis_accepted_interactions.jsonl` and runtime configuration. It may consult previous products for diagnostics.
Outputs	`ObserverBridge/observer_bridge_candidates.jsonl`, `ObserverBridge/observer_bridge_ranked_events.jsonl`, JSON/CSV summaries, camera-view and overlay diagnostics.
Invariant policy	Number of candidates scored equals number of accepted DIS interactions consumed. FOV is not used as a destructive filter.
Approximation	Geometric escape, visibility, FOV, redshift, and line-of-sight proxies. No secondary-particle generation.

Chapter 8

POWHEG Dry Run / Hard Process Preparation

8.1 Dry-run role

H3-W9a prepares local POWHEG DIS jobs but deliberately does not execute `pwhg_main`. It consumes ranked Observer Bridge events and writes its own products under `POWHEG/`. It does not modify `ObserverBridge/`, `DIS/`, `ForwardGeodesics/`, or `UHEsource/`.

The implemented run mode is

$$\texttt{powheg\_run\_mode} = \texttt{dry\_run}. \tag{8.1}$$

The reports record

$$\texttt{powheg\_invoked} = \texttt{false}, \tag{8.2}$$

$$\texttt{pwhg\_main\_executed} = \texttt{false}, \tag{8.3}$$

$$\texttt{powheg\_lhe\_generated} = \texttt{false}. \tag{8.4}$$

8.2 Candidate selection

The driver reads `ObserverBridge/observer_bridge_ranked_events.jsonl`, sorts candidates by decreasing final observation score, and uses the original input order as a tie-breaker. The implemented selection policies are:

- `top_score`: take highest-score candidates up to `max_powheg_events`.
- `score_threshold`: discard candidates below `min_final_observation_score`, then cap at `max_powheg_events`.
- `all_candidates`: represented by the same sorted traversal and cap.

8.3 Input-card mapping

Each selected candidate receives a request identifier of the form `H3PWHG-000001`. Each request has its own `powheg_input_cards/<request_id>/powheg.input`. The implemented mapping is

$$E_{\nu,\mathrm{local}} \rightarrow \texttt{ebeam1}, \tag{8.5}$$

$$m_N = 0.938272\ \mathrm{GeV} \rightarrow \texttt{ebeam2}, \tag{8.6}$$

$$\texttt{events\_per\_candidate} \rightarrow \texttt{numevts}, \tag{8.7}$$

$$\texttt{channel\_type} = 3, \tag{8.8}$$

$$\texttt{vtype} = 2. \tag{8.9}$$

The dry-run card also records fixed PDF choices `lhans1 = 303400` and `lhans2 = 303400`. The implemented scale cap is

$$Q_{\max} = \min\left(2\sqrt{0.938272\,E_{\nu,\mathrm{local}}},\ 10^5\right). \tag{8.10}$$

For the implemented seed mode, the seed is

$$\texttt{powheg\_seed} = \texttt{random\_seed} + \texttt{candidate\_rank}. \tag{8.11}$$

8.4 Event requests

The event request file `POWHEG/powheg_event_requests.jsonl` records for each prepared job:

$$\begin{aligned}\mathcal{R}_{\mathrm{POWHEG}} = \{&\texttt{powheg\_request\_id}, \texttt{candidate\_rank}, \texttt{interaction\_id}, \texttt{event\_id}, \texttt{source\_sample\_id},\\ &E_{\nu,\mathrm{local}}, w_{\mathrm{physics}}, w_{\mathrm{observer}}, S_{\mathrm{obs}}, \texttt{powheg\_input\_path}, \texttt{powheg\_seed}, \texttt{powheg\_status}\}.\end{aligned} \tag{8.12}$$

The status is `dry_run_ready`; `powheg_invoked` is false for every request.

8.5 Inputs and outputs

Role	Implemented item
Inputs	`ObserverBridge/observer_bridge_ranked_events.jsonl` and POWHEG configuration values.
Outputs	`POWHEG/powheg_event_requests.jsonl`, input cards, summary JSON/CSV, report JSON, and diagnostic plots.
No output yet	No `pwgevents.lhe`; no PYTHIA; no GEANT4; no photon transport.
Backend	C++17 executable `bin/hadros3_powheg_driver`. Python handles orchestration and plots.

Chapter 9

Provenance and Statistical Weights

9.1 Stage provenance

Each stage records the backend, products, validation flags, and disabled future stages relevant to its current implementation. A typical provenance record contains:

$$\{\text{parameters}, \text{products}, \text{validation}, \text{stage summaries}, \text{disabled stages}\}. \tag{9.1}$$

The implemented provenance explicitly marks future expensive stages as disabled until they are introduced:

$$\texttt{pythia\_invoked} = \texttt{false}, \tag{9.2}$$

$$\texttt{geant4\_invoked} = \texttt{false}, \tag{9.3}$$

$$\texttt{photon\_transport\_invoked} = \texttt{false}, \tag{9.4}$$

$$\texttt{expensive\_event\_generation\_invoked} = \texttt{false}. \tag{9.5}$$

For H3-W9a, POWHEG is active only as dry-run preparation:

$$\texttt{status} = \texttt{powheg\_jobs\_prepared\_no\_lhe}. \tag{9.6}$$

9.2 Weight chain

The implemented source and direction samplers are configured so that the default source and direction weights are unity:

$$w_{\text{source}} = 1, \tag{9.7}$$

$$w_{\text{direction}} = 1. \tag{9.8}$$

The DIS sampler records interaction and pre-event weights. The Observer Bridge then uses

$$w_{\text{physics}} = \texttt{final\_pre\_event\_weight} \tag{9.9}$$

and computes

$$S_{\text{obs}} = w_{\text{physics}} w_{\text{observer}}. \tag{9.10}$$

The POWHEG dry-run carries those weights into each event request; it does not change the physics score and does not generate a hard event yet.

9.3 Random seeds

The implemented random stages record seeds:

- UHE source sampling records its source seed.
- DIS interaction sampling records its acceptance and point-sampling seed.
- POWHEG dry-run records `random_seed` and the per-candidate `powheg_seed`.

These seeds are part of the statistical contract of the pipeline.

Chapter 10

Pipeline Summary

10.1 Implemented stage contract

The current implemented pipeline can be summarized as

$$\text{UHESource} \rightarrow \text{ForwardGeodesics} \rightarrow \text{DIS} \rightarrow \text{ObserverBridge} \rightarrow \text{POWHEG DryRun}. \quad (10.1)$$

The camera preview is an observer-view diagnostic that can be used by the dashboard and by Observer Bridge overlays, but it is not a physical secondary-particle propagation stage.

Table 10.1: Implemented pipeline products and semantic status.

Stage	Official inputs	Official outputs	Status
UHE Source	Configuration	`UHEsource/` JSONL/JSON	Sampling proxy with monoenergetic source.
Forward Geodesics	`UHEsource/`	`ForwardGeodesics/` paths and segments	Kerr null geodesic propagation.
DIS Sampler	`ForwardGeodesics/`, `UHEsource/`	`DIS/` optical-depth and accepted interactions	Optical-depth DIS sampler.
Observer Bridge	`DIS/dis_accepted_interactions.jsonl`	`ObserverBridge/` candidates and ranked events	Non-destructive geometric scoring proxy.
POWHEG Dry Run	`ObserverBridge/observer_bridge_ranked_events.jsonl`	`POWHEG/` generated input cards	Prepared jobs only; no LHE.

10.2 Explicit non-goals of the implemented stages

The current pipeline does not yet implement:

- real POWHEG event execution inside the main pipeline;
- LHE generation from H3-W9a;
- PYTHIA showering and hadronization;
- GEANT4 detector or material transport;

- photon transport from secondaries to the observer;
- full GRMHD torus evolution;
- full physical visibility, escape, redshift, or line-of-sight transport.

Those future stages must preserve the contract that each stage consumes official previous outputs, writes only to its own directory, records provenance, and marks expensive or proxy physics explicitly.

Appendix A

Appendix: Symbols and Product Names

Product or field	Meaning
`uhe_neutrino_source_samples.jsonl`	UHE source positions, energies, directions, and sampling weights.
`uhe_neutrino_forward_path_segments.jsonl`	Forward-going chronologic path segments with midpoint coordinates, momenta, and segment lengths.
`dis_path_optical_depths.jsonl`	Path-level DIS optical depths and interaction probabilities.
`dis_accepted_interactions.jsonl`	Accepted DIS interactions and their density-checked interaction points.
`observer_bridge_candidates.jsonl`	All accepted DIS interactions scored as observation candidates.
`observer_bridge_ranked_events.jsonl`	Same candidate family ranked by final observation score.
`powheg_event_requests.jsonl`	Dry-run POWHEG event requests and card paths.
`final_pre_event_weight`	Physics weight consumed by Observer Bridge as implemented.
`final_observation_score`	Product of implemented physics and observer weights.
`powheg_invoked`	False in H3-W9a dry run.
`pwhg_main_executed`	False in H3-W9a dry run.
`powheg_lhe_generated`	False in H3-W9a dry run.